CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 11: Implement Prompt Reuse

Exam objective: Implement prompt reuse through prompt caching, modular prompt architecture, reusable components, version management, Claude Skills with progressive loading, and the separation of static versus dynamic context.

Q1 **A financial services firm's Claude-powered assistant sends the same 8,000-token system prompt (tone, compliance guardrails, tool definitions) on every single request, with only the user's question changing each time. What is the BEST way to reduce redundant cost without changing the system prompt's content?**

A Shorten the system prompt by removing all guardrails

B **Use prompt caching so the stable, unchanged portion of the prompt is cached and reused across requests rather than being fully reprocessed every time**

C Move the system prompt into the user turn instead

D Switch to a different model that ignores system prompts

CORRECT ANSWER **B**

WHY Prompt caching is designed exactly for this pattern: a large, stable prefix reused across many requests can be cached to cut cost and latency, without altering its content. Removing guardrails sacrifices safety for cost, moving stable content into the user turn breaks the role separation and likely defeats caching eligibility, and switching models to "ignore" system prompts isn't a real feature or sound design.

Q2 **An architect designing a prompt-caching strategy for a customer support agent must decide how to order content within the prompt to maximize cache effectiveness. What is the BEST structuring principle?**

A Put the most frequently changing content (e.g., the live user message) first, followed by stable instructions

B **Put the most stable, unchanging content (system instructions, guardrails, static tool definitions) first/early, and place variable, per-request content (user messages, retrieved context) later**

C Randomize the order of prompt sections on every request

D Order has no effect on caching behavior

CORRECT ANSWER B

WHY Caching mechanisms typically cache a stable prefix, so placing unchanging content first and variable content last maximizes the portion of the prompt that can be reused across requests. Putting variable content first breaks the stable prefix and defeats caching, randomizing order guarantees cache misses, and prompt structure does materially affect caching efficiency.

Q3 **A manufacturing company maintains one "guardrails and tone" prompt component, one "defect-triage task" component, and one "safety-escalation task" component, which are composed together differently for different features. What architectural pattern does this best describe?**

A Monolithic prompting, where every feature has one giant undifferentiated prompt

B **Modular prompt architecture, where reusable components are composed together to build different feature-specific prompts**

C Zero-shot prompting

D Prompt caching exclusively, with no other architectural benefit

CORRECT ANSWER B

WHY Building prompts from independently maintained, reusable components that are composed per feature is the definition of modular prompt architecture, improving maintainability and consistency. This is the opposite of monolithic prompting, it is not a prompting technique like zero-shot, and while it can support caching, caching alone doesn't describe this compositional structure.

Q4 A legal tech company ships three versions of its contract-review system prompt over six months (v1.0, v1.1, v2.0), each with behavioral changes. A client reports that outputs seem to have changed and asks exactly what changed and when. What does this scenario demonstrate the need for?

A Avoiding any further changes to the prompt going forward

B Rigorous prompt version management: tracked version numbers, changelogs, and the ability to map any historical output back to the exact prompt version that generated it

C Deleting old versions once a new one ships

D Telling the client that prompt changes are untrackable by nature

CORRECT ANSWER B

WHY Proper version management—tracked versions, changelogs, and traceability from output back to the exact prompt version—is exactly what allows an architect to answer "what changed and when" with confidence. Freezing the prompt forever blocks legitimate improvement, deleting old versions destroys the audit trail, and claiming changes are untrackable is both false and a governance failure.

Q5 An enterprise deployment uses Claude Skills so that a general-purpose assistant can access specialized capabilities (e.g., spreadsheet analysis, PDF generation) only when needed, rather than always having every specialized instruction loaded. What benefit does this progressive loading design PRIMARILY provide?

A. It guarantees the assistant will never make mistakes

B. It keeps the baseline context lean by only pulling in detailed skill-specific instructions/resources when the relevant skill is actually invoked, rather than bloating every request with all possible specialized instructions

C. It eliminates the need for any system prompt

D. It requires every skill to be rewritten for each individual request

CORRECT ANSWER B

WHY Progressive loading is a context-efficiency pattern: specialized instructions are only loaded on demand, keeping unrelated requests lean. It does not guarantee correctness, a system prompt is still needed for baseline behavior/guardrails, and skills are defined once and loaded as needed rather than rewritten per request.

Q6 **A retail company wants to A/B test two variants of a product-recommendation system prompt in production to measure which produces better click-through rates. What is the BEST practice to support this safely?**

A Alternate between the two variants randomly with no tracking of which variant produced which output

B **Tag each request/response with the exact prompt version used, store this alongside outcome metrics, and roll out changes behind a controlled version flag so results can be attributed correctly**

C Edit the live prompt directly in production without recording any version history

D Only test one variant at a time with no comparison baseline, then guess which was better

CORRECT ANSWER **B**

WHY Reliable experimentation requires attributing outcomes to specific prompt versions via tagging and controlled rollout, which is a core practice of prompt version management applied to testing. Untracked random alternation prevents valid attribution, untracked live edits destroy the ability to reproduce results, and testing without a comparison baseline isn't a valid A/B test.

Q7 **An architect is separating "static" and "dynamic" context in a prompt for an insurance underwriting assistant: company underwriting guidelines (rarely change) versus the specific applicant's data (changes every request). What is the BEST reason to keep these separated and structured distinctly?**

A Static content should always be discarded to save tokens

B **Separating static (cacheable, reusable) content from dynamic (per-request) content allows caching benefits to apply to the static portion while keeping the system flexible for varying applicant data**

C Dynamic content should be hardcoded into the static guidelines to simplify the prompt

D There is no benefit; static and dynamic content should always be merged into one undifferentiated block

CORRECT ANSWER **B**

WHY Explicitly separating static from dynamic content is what enables prompt caching to be effective on the stable portion while still correctly handling varying per-request data. Discarding static content loses necessary guidelines, hardcoding dynamic data into static content breaks reuse, and merging everything undifferentiated prevents caching optimization and reduces maintainability.

Q8 **A SaaS company has built five different Claude-powered features, each re-implementing its own version of a "cite your sources and never fabricate" instruction block, each written independently with slightly different wording. What risk does this duplication create?**

A. No risk; duplicated independent wording is always equally effective

B. Inconsistent behavior across features and a maintenance burden where fixing a discovered flaw in the instruction requires updating five separate, divergent copies instead of one shared component

C. Improved performance, since more independently-worded instructions always reinforce each other

D. Automatic version synchronization across all five features

CORRECT ANSWER B

WHY Duplicated, independently-maintained instruction blocks drift over time and multiply the effort (and risk of missed updates) needed to fix a shared flaw, which is precisely the problem reusable/modular components solve. There is real risk from inconsistency, duplication does not improve performance, and nothing in this setup provides automatic synchronization.

Q9 **An architect configuring prompt caching for a high-traffic support bot notices that even though the system prompt is identical across requests, cache hits are inconsistent. Investigation reveals a timestamp is being inserted at the very start of the system prompt on every request. What is the BEST fix?**

A. Leave the timestamp at the start since caching is unaffected by content changes

B. Move the timestamp (and any other frequently changing values) out of the cached prefix—e.g., to the end of the prompt or into the user turn—so the stable prefix remains byte-identical across requests and can be cached effectively

C. Remove caching entirely since it clearly does not work

D. Insert a different timestamp format to fix the issue

CORRECT ANSWER B

WHY Caching typically depends on the cached prefix being identical across requests; placing frequently changing values (like timestamps) early in that prefix invalidates the cache on every call, so moving them out of the stable prefix restores cache effectiveness. Content changes absolutely affect caching, abandoning caching discards a valuable optimization unnecessarily, and merely changing the timestamp's format doesn't address the root cause (its placement).

Q10 **A telecom company builds a library of reusable prompt "snippets" (e.g., a standard escalation-policy block, a standard tone block, a standard JSON-output block) that different teams assemble into their own feature prompts. What governance practice is MOST important to add as this library grows?**

A. Allow any team to silently edit shared snippets without review, since speed matters most

B. Establish ownership, review, and versioning for shared snippets, since an uncoordinated change to a widely-reused snippet could unexpectedly affect many downstream features at once

C. Prevent any team from reusing snippets to avoid coordination overhead entirely

D. Require every team to copy-paste snippets into their own prompts instead of referencing a shared source

CORRECT ANSWER B

WHY Because shared, reusable components fan out to many consumers, uncoordinated edits can have wide, unexpected blast radius—making ownership, review, and versioning essential governance. Allowing silent edits is risky, banning reuse forfeits the benefits of modular architecture, and copy-pasting instead of referencing a shared source reintroduces the duplication/drift problem reuse is meant to solve.

Q11 **An architect is deciding whether a newly built "financial risk analysis" capability should be implemented as an always-loaded part of the main system prompt or as a Claude Skill loaded only when relevant. The capability is used by fewer than 5% of requests but requires substantial specialized instructions. What is the BEST choice?**

A. Always load it in the main system prompt so it's instantly available for every request

B. Implement it as a Skill with progressive loading, so its substantial instructions only consume context when that specific capability is actually invoked

C. Build it as a completely separate, unrelated application with no shared context management

D. Refuse to build the capability since it's rarely used

CORRECT ANSWER B

WHY For a low-frequency but context-heavy capability, progressive loading (Skills) avoids bloating every request's baseline context while still making the capability available on demand. Always loading it wastes tokens on the 95% of requests that don't need it, building a fully separate unrelated application ignores a more efficient native pattern, and refusing to build it ignores a legitimate business need.

Q12 **A government agency wants to roll back a recently deployed prompt change after discovering it caused an increase in overly verbose responses. What does this scenario underscore the importance of?**

A Never deploying prompt changes without months of advance notice

B **Maintaining prompt version history with the ability to quickly revert to a known-good prior version, rather than only having the current live prompt available**

C Permanently disabling the feature instead of reverting

D Rewriting the prompt from memory since no version history exists by default

CORRECT ANSWER B

WHY The ability to quickly roll back to a known-good version depends on having maintained version history, which is a core benefit of disciplined prompt version management. Requiring months of notice is impractical for quick fixes, disabling the feature is an overreaction when a simple revert suffices, and relying on memory instead of stored history is an avoidable risk that good versioning practice prevents.

Q13 **A SaaS company's engineering team wants to reuse the same "output must be valid JSON matching schema X" instruction block across 12 different features. Which implementation approach BEST supports long-term maintainability?**

A Paste the instruction block, verbatim, into all 12 feature prompts as independent copies

B **Define the instruction block once as a shared, versioned component and reference/include it wherever needed, so future schema updates are made in a single place**

C Let each team write it slightly differently based on their own interpretation

D Avoid structured output requirements altogether to sidestep the reuse question

CORRECT ANSWER **B**

WHY A single source-of-truth component referenced across features is the maintainable approach: a future schema change updates all consumers from one place. Copy-pasting into 12 locations creates drift risk, letting teams reinterpret the instruction independently undermines consistency, and avoiding structured output entirely sidesteps the actual business requirement rather than solving the reuse problem.

Q14 An architect notes that prompt caching provides the largest benefit when a prompt prefix is (a) large, (b) reused frequently across many requests, and © unchanged between those requests. Given this, which scenario benefits LEAST from prompt caching?

A. A stable 10,000-token system prompt used by thousands of requests per hour with no per-request changes to that prefix

B. A one-off, single-use prompt that is never reused and contains almost entirely unique, request-specific content

C. A tool-definition block reused identically across all requests for a given agent

D. A reusable guardrails block shared and reused unchanged across many features

CORRECT ANSWER B

WHY Caching benefits depend on reuse and stability; a one-off prompt with unique content, used exactly once, has no opportunity to be reused from cache and thus gains essentially no benefit. The other options describe large, stable, frequently-reused prefixes, which are precisely the conditions where caching provides the most value.

Q15 A manufacturing company's architecture review asks whether Claude Skills and prompt caching are competing or complementary techniques for prompt reuse. What is the MOST accurate answer?

A. They are mutually exclusive; a deployment must choose only one

B. They are complementary: Skills manage which specialized instructions/resources get loaded into context and when (progressive loading), while prompt caching optimizes the cost/latency of reusing stable content that is already present in context across requests

C. Skills replace the need for a system prompt, while caching replaces the need for Skills

D. Caching only works with few-shot examples and has no relationship to system prompts or Skills

CORRECT ANSWER B

WHY Skills address what context gets loaded and when (reducing unnecessary context on low-relevance requests), while caching addresses the cost/latency of reusing whatever stable content is present—these operate at different layers and work well together. They are not mutually exclusive, neither replaces the system prompt, and caching applies broadly to stable prompt content, not only few-shot examples.

Q16 **An architect wants to maximize prompt-cache hit rates for a support agent whose request is built from tool definitions, a system prompt, and conversation messages. A teammate proposes adding one new optional tool definition mid-week to support a new feature. What should the architect expect regarding caching?**

A Caching is unaffected by tool definition changes since tools are cached completely independently of the system prompt and messages

B **Cache prefixes are built in a fixed order—tools, then system prompt, then messages—so a change at any level invalidates that level and everything after it; because tools come first, modifying the tool definitions invalidates the entire cache, including the system prompt and message history portions**

C Only the newly added tool's own definition is affected; all previously cached content remains valid

D Adding a tool definition has no effect on caching because caching only applies to user messages

CORRECT ANSWER **B**

WHY Prompt caching is built as an ordered prefix of tools, then system prompt, then messages, so a change at the tools level invalidates not just that segment but everything that follows it in the order, meaning a single new tool definition can invalidate the whole cache. Caching is not independent per layer, the blast radius is not limited to just the new tool, and caching is not restricted to user messages only.

Q17 An architect is deciding on a prompt-caching configuration for a large, stable 15,000-token system prompt used by an agent whose requests to the same conversation are typically more than 20 minutes apart, but less than an hour apart. Which configuration detail is MOST relevant to this decision?

A Cache breakpoints always cost extra regardless of how many are used, so minimizing breakpoints is the only lever available

B **The default cache lifetime is 5 minutes, refreshed at no extra cost each time it's used, while a 1-hour lifetime is available at twice the base input token price to write; since this workload's gap between reuses (20-60 minutes) exceeds the 5-minute default, the 1-hour option is worth evaluating against its extra write cost for this large, stable prefix**

C Prompt caching has no concept of lifetime; once written, a cache entry persists indefinitely until manually cleared

D Caching reduces the cost of generating output tokens, so the main benefit here is faster report generation

CORRECT ANSWER **B**

WHY Documented cache behavior gives a 5-minute default lifetime (refreshed free on reuse) and an optional 1-hour lifetime at double the base input price to write; a request gap of 20-60 minutes would miss the 5-minute window entirely, making the 1-hour option worth weighing against its added write cost for a large, reused prefix. Breakpoints themselves add no extra cost, cache entries are not indefinite, and caching has no effect on output token cost.

Q18 **A platform team publishes a new version of a widely used "contract-analysis" Claude Skill to add a bug fix, and several production agents reference that Skill. One engineer uploads only the changed files for the new version, assuming unchanged files carry over automatically from the previous version. What is the MOST important problem with this approach, and what should production deployments do instead?**

A There is no problem; Skills automatically merge new uploads with the previous version's files

B **A new Skill version is a complete snapshot, not a delta—any files left out are not carried over from the previous version, which is a data-loss risk; production deployments should also pin a specific Skill version rather than using "latest," since an unpinned version can change what deployed agents run the moment anyone publishes a new version**

C Skills cannot be versioned at all, so this scenario is not possible

D Only the Skill's description needs version control; the underlying files are never versioned

CORRECT ANSWER **B**

WHY A new Skill version must be uploaded as the full, complete file set because it replaces the entire snapshot rather than patching it—omitting files causes them to be dropped, not carried over—and production should pin an explicit version so that someone else publishing a new version does not silently change already-deployed agent behavior. Skills do not auto-merge across versions, they are versioned as a whole artifact, and the files themselves (not just the description) are versioned. --